

CyberCase RAG Service — สถาปัตยกรรม & คู่มืออ้างอิงโค้ดทุกฟังก์ชัน

เอกสารนี้เขียนขึ้นใหม่ทั้งหมดจากการอ่านซอร์สโค้ด `rag_service/` ทุกไฟล์ (ไม่อ้างอิงเอกสารเดิม) ครอบคลุม: ภาพรวมสถาปัตยกรรม, เทคนิค/method, DB schema, โมเดลทุกตัวใน pipeline, และคำอธิบาย **ทุกฟังก์ชัน/คลาส** แยกตามไฟล์

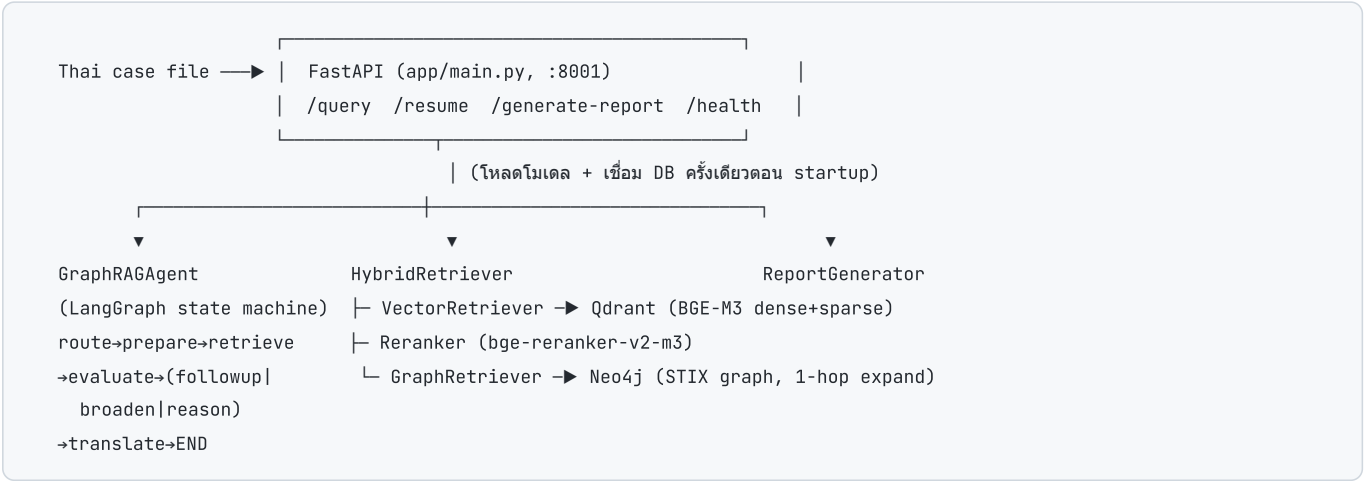
ขอบเขต: `rag_service/app/RAG/GraphRAG` (pipeline RAG หลัก), `rag_service/app` (FastAPI + CLI + utilities), `rag_service/docs` , และ `rag_service/finetune` (โมดูล fine-tune MITRE specialist)

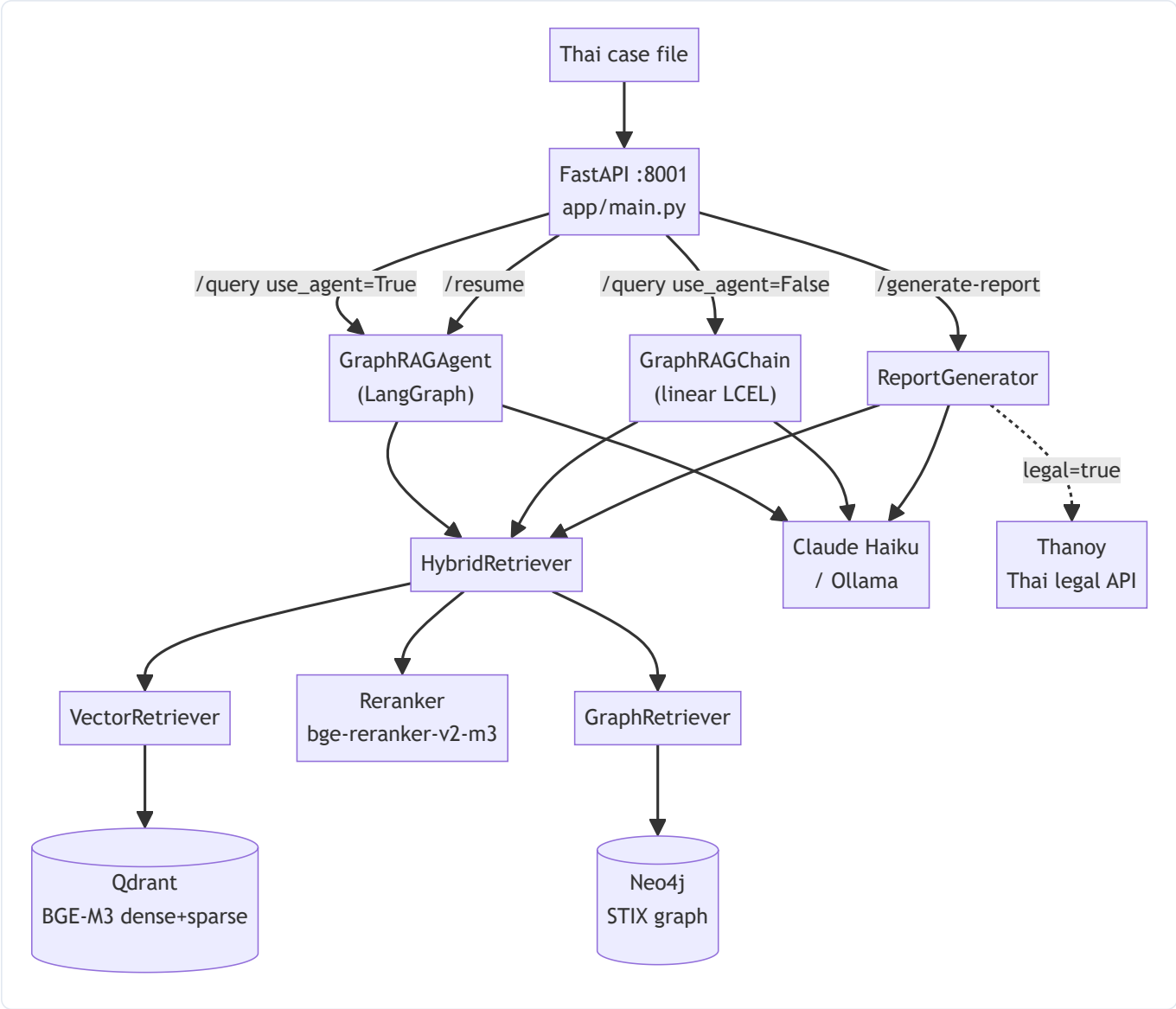
สารบัญ

- 1. ภาพรวมสถาปัตยกรรม
- 2. เทคนิค/Method ที่ใช้
- 3. โมเดลทุกตัวใน Pipeline
- 4. DB Schema (Neo4j + Qdrant)
- 5. โครงสร้างไดเรกทอรี
- 6. Request Lifecycle
- 7. อ้างอิงโค้ดทุกฟังก์ชัน - 7.1 app/main.py - 7.2 config.py - 7.3 models.py - 7.4 pipeline/ - 7.5 retrieval/ - 7.6 ingestion/ - 7.7 evaluation/ - 7.8 CLI & utilities - 7.9 finetune/

1. ภาพรวมสถาปัตยกรรม

`rag_service` เป็น **FastAPI microservice (พอร์ต 8001)** ที่โฮสต์ GraphRAG pipeline ทั้งหมด Backend gateway (พอร์ต 8000) เป็นเพียง proxy ที่เรียกบริการนี้ผ่าน HTTP ส่วน RAG ทั้งหมดอยู่ที่นี้





3 เส้นทางหลักของบริการ

Endpoint	คลาสที่ใช้	เส้นทาง
POST /query (use_agent=True)	GraphRAGAgent	agentic LangGraph: decompose → quota-retrieve → evaluate → (follow-up/broaden) → reason → translate
POST /query (use_agent=False)	GraphRAGChain	linear LCEL: translate → dual-query retrieve → reason → translate
POST /resume	GraphRAGAgent.resume	ดำเนิน session ที่ pause เพื่อถาม follow-up ต่อ
POST /generate-report	HybridRetriever + ReportGenerator (+ thanoy_client)	รายงานคดี 3 ส่วน: case summary + faithful MITRE table + (optional) Thai legal

สอง pipeline ที่ขนานกันในโค้ด - GraphRAGAgent (pipeline/agent_graph.py) — เส้นทาง production agentic ใหม่ (decomposer + quota + self-reflection + follow-up) - GraphRAGChain (pipeline/chain.py) — เส้นทาง linear เดิม (ยังใช้ใน /generate-report เพื่อแปล query และใน eval generation)

2. เทคนิค/Method ที่ใช้

เทคนิค	ที่อยู่	สาระสำคัญ
GraphRAG	<code>retrieval/hybrid_retriever.py</code>	รวม Vector search + Graph expansion: ดึง top-K จาก Qdrant แล้วขยาย subgraph จาก Neo4j ตาม stix_id
Hybrid Vector Search (Dense + Sparse)	<code>vector_retriever.py</code>	BGE-M3 ให้ทั้ง dense (1024-d) และ sparse (lexical) → Qdrant native RRF fusion ในเครื่อง query เดียว
RRF (Reciprocal Rank Fusion)	Qdrant <code>FusionQuery(Fusion.RRF)</code> + config <code>RRF_K=60</code>	รวมผล dense/sparse และผลข้าม collection
Cross-encoder Reranking	<code>retrieval/reranker.py</code>	<code>bge-reranker-v2-m3</code> ให้คะแนน (query, doc) ใหม่ → sigmoid → top-K ก่อนป้อน graph + LLM
Node-type re-weighting	<code>hybrid_retriever._reweight_by_type</code>	คูณคะแนน Technique/Subtechnique/Tactic ขึ้น, Group/Software/Campaign ลง → technique ลอยขึ้น
Cross-lingual retrieval	<code>pipeline/cross_lingual.py</code>	input ไทยเสมอ, output ไทยเสมอ; แปลอังกฤษเป็น internal-only
Dual-query (chain/report path)	<code>build_retrieval_queries</code>	ดึงทั้ง query อังกฤษที่แปล + query ไทยต้นฉบับขนานกัน กัน mistranslation ทำ retrieval พัง
Query Decomposition (agent path)	<code>pipeline/query_decomposer.py</code>	แตก incident เป็น sub-query atomic ต่อ technique (ภาษาเดิม, ไม่แปล — BGE-M3 multilingual)
Per-query Quota Retrieval	<code>hybrid_retriever.retrieve_multi_quota</code>	เก็บ top- <code>per_query_k</code> ของแต่ละ sub-query แล้ว round-robin interleave → ทุก technique ได้พื้นที่
Self-reflection / Self-RAG loop	<code>pipeline/evaluator.py</code> + agent edges	LLM ประเมิน context พอหรือไม่ → SUFFICIENT / INSUFFICIENT(ถาม follow-up) / BROADEN_SEARCH
Slot-aware Follow-up	<code>evaluator.py</code> + <code>agent_graph._resume_with_answer</code>	ถามทีละ slot (initial access → credential_theft → priv_esc → lateral → impact) ไม่ถามซ้ำ
Query Merger	<code>pipeline/query_merger.py</code>	รวม query เดิม + คำตอบ follow-up เป็น query เดียวที่ MITRE-aligned
3-stage cross-lingual generation	<code>cross_lingual</code> prompts	(1) translate query → EN, (2) reasoning LLM → simplified EN, (3) translation LLM → Thai
Faithful MITRE table	<code>report_generator.extract_mitre_entities</code>	สร้างตารางจาก entity ที่ retrieve จริง (ไม่ใช่จาก LLM) → ID ไม่ถูก hallucinate
CJK Thai-only guard	<code>report_generator._sanitize_thai</code>	ตรวจ token จีน/ญี่ปุ่น/เกาหลีหลุดในรายงาน → re-translate field เป็นไทยล้วน
Domain filter (mobile กัน ปน)	<code>vector_retriever.search_entities</code> + <code>config.ATTACK_DOMAIN_FILTER</code>	กรอง entity ให้เหลือ domain enterprise หลัง retrieval
Agentic state machine	<code>pipeline/agent_graph.py</code> (LangGraph)	StateGraph: node + conditional edges + in-memory session store สำหรับ pause/resume
Thai legal delegation	<code>pipeline/thanoy_client.py</code>	ไม่สอนกฎหมายไทยให้ MITRE model (กัน hallucinate มาตรา) → เรียก Thanoy API แทน
Eval: retriever metrics	<code>evaluation/retriever_metrics.py</code>	Hit@K, Recall@K, Precision@K, MRR, NDCG@K, MAP
Eval: generation metrics	<code>evaluation/generation_metrics.py</code>	RAGAS (faithfulness, answer_correctness) + fallback Token-F1/ROUGE-L/BERTScore
Graph-grounded eval dataset	<code>evaluation/generate_eval_dataset.py</code>	สร้าง ground-truth จาก Cypher (กราฟ = ground truth) → ไม่ label มือ
MITRE specialist fine-tune	<code>finetune/</code>	QLoRA/16-bit LoRA บน Qwen, dataset จาก STIX, A/B เทียบ base vs fine-tune ผ่าน env-swap

3. โมเดลทุกตัวใน Pipeline

บทบาท	โมเดล (cloud)	โมเดล (local <code>--local</code>)	ตั้งค่าที่
Embedding	BAAI/bge-m3 (1024-d, FP16, dense+sparse)	(เหมือนกัน)	EMBED_MODEL , USE_FP16
Reranker	BAAI/bge-reranker-v2-m3 (cross-encoder, รองรับไทย)	(เหมือนกัน)	RERANKER_MODEL
Reasoning / Translation / Router / Decomposer LLM	claude-haiku-4-5	qwen2.5:7b (Ollama)	LLM_MODEL / LOCAL_LLM_MODEL
Evaluator / Query-merger LLM	claude-haiku-4-5	gemma3:4b (Ollama)	EVALUATOR_LLM_MODEL / LOCAL_EVAL_MODEL
RAGAS judge (eval)	Claude Haiku → OpenRouter qwen/qwen-2.5-72b-instruct	gemma3:4b	RAGAS_LLM_MODEL , OPENROUTER_*
RAGAS embeddings (eval)	nomie-embed-text (Ollama, local)	(เหมือนกัน)	hard-coded ใน generation_metrics
Thai legal AI	Thanoy (iApp REST API)	(เหมือนกัน)	THANOY_API_*
Fine-tune target	—	Qwen/Qwen3.5-4B → mitre-qwen3.5:4b (16-bit LoRA)	finetune/ft_config.py

หมายเหตุ: master switch `USE_LOCAL` (env) สลับทั้ง pipeline ไป Ollama; CLI ใช้ flag `--local` (eval/finetune) `download_model.py` (ใช้ตอน build Docker) ยัง pre-cache reranker ตัวเก่า `cross-encoder/mmarco-mMiniLMv2-L12-H384-v1`

Tech stack (จาก `requirements.txt`): FastAPI + Uvicorn, Pydantic v2, `neo4j` , `qdrant-client` , `FlagEmbedding` (BGE-M3), `sentence-transformers` (reranker), LangChain (`langchain-anthropic` , `langchain-ollama` , `langgraph`), `anthropic` , `httpx` , `stix2` , RAGAS/datasets/bert-score/rouge-score (eval), torch/transformers. **Deploy:** `Dockerfile` (python:3.11-slim) ติดตั้ง torch CPU, pre-cache embedding model, รัน `uvicorn app.main:app --port 8001` . Neo4j + Qdrant เป็น cloud-hosted

4. DB Schema

4.1 Neo4j (Graph DB) — STIX entities + relationships

Node labels (ทุก node ได้ base label `:Entity` เพิ่มด้วย เพื่อ match เร็วตอนสร้าง edge):

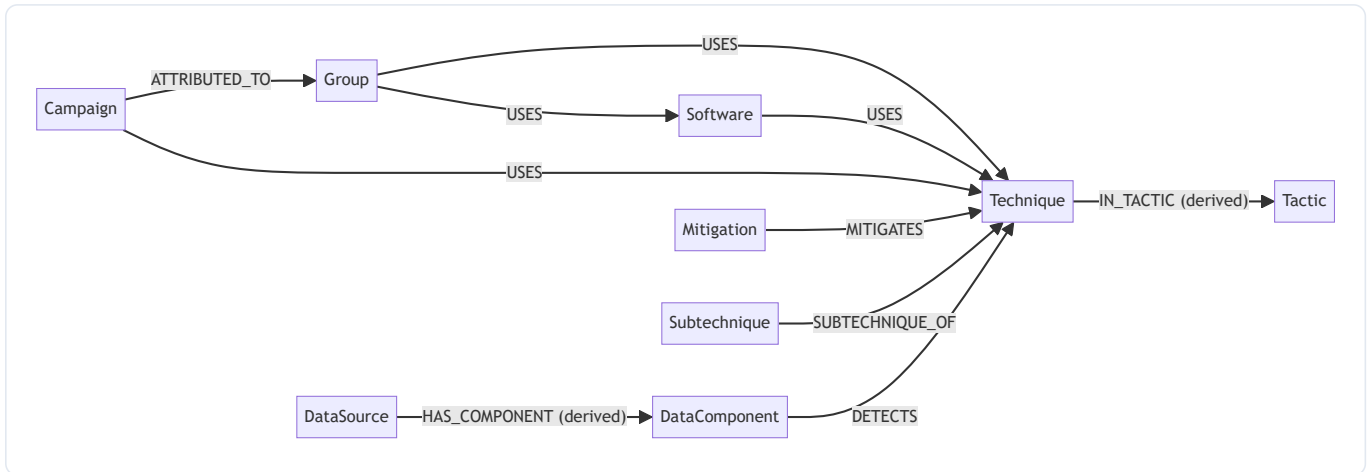
`Technique` , `Subtechnique` , `Group` , `Software` , `Campaign` , `Mitigation` , `Tactic` , `DataSource` , `DataComponent`

Node properties (จาก `graph_loader._entity_to_props`) : - ทุก node: `stix_id` (unique), `attack_id` , `name` , `description` (≤5000 ตัวอักษร), `url` , `domain` (`enterprise` / `mobile` / `ics`) - `Technique` / `Subtechnique` : `platforms` , `is_subtechnique` - `Software` : `software_type` (`tool` / `malware`), `aliases` - `Tactic` : `shortname` (เช่น `initial-access`) - `Group` / `Campaign` : `aliases`

Edge types (จาก `RELATIONSHIP_TYPE_MAP` + derived edges):

Edge	จาก → ไป	ที่มา
USES	Group/Software/Campaign → Technique/Software	STIX <code>uses</code>
MITIGATES	Mitigation → Technique	STIX <code>mitigates</code>
SUBTECHNIQUE_OF	Subtechnique → Technique	STIX <code>subtechnique-of</code>
ATTRIBUTED_TO	Campaign → Group	STIX <code>attributed-to</code>
DETECTS	DataComponent → Technique	STIX <code>detects</code>
IN_TACTIC	Technique → Tactic	derived จาก <code>kill_chain_phases</code>
HAS_COMPONENT	DataSource → DataComponent	derived จาก <code>x_mitre_data_source_ref</code>
(ข้าม REVOKED_BY)	—	กรองทิ้ง

Edge property: `stix_id`, `description` (≤ 5000)



Constraints: `stix_id` IS UNIQUE ทุก label + `:Entity` **Indexes:** `Technique/Subtechnique.attack_id`, `Group/Software.name`, `Tactic.shortname`

4.2 Qdrant (Vector DB) — BGE-M3 embeddings

2 collections: - `mitre_entities` (~2,733 docs): ข้อความ embed = `"{node_label}: {name}. {description}"` - `mitre_relationships` (~25,467 docs): ข้อความ embed = `"{source_name} {edge_label} {target_name}: {description}"`

Vector config (ต่อ point): `dense` (size 1024, Cosine) + `sparse` (SparseVector) **Point ID:** UUID ที่ derive จาก `stix_id` (`uuid_from_stix_id`)

Payload schema: - entities: `stix_id`, `attack_id`, `entity_type="Node"`, `node_label`, `name`, `domain`, `url`, `document` - relationships: `stix_id`, `entity_type="Relationship"`, `edge_label`, `source_id`, `target_id`, `source_name`, `target_name`, `document`

⚠ payload `domain` มีเฉพาะ collection entities (relationships ไม่มี) — domain filter จึงทำได้กับ entity เท่านั้น ⚠ การ filter `domain` ฝั่ง Qdrant ต้องมี payload index (cloud ปัจจุบันไม่มี) → โค้ดจึง over-fetch แล้วกรองใน Python

5. โครงสร้างไดเรกทอรี

```
rag_service/
├── Dockerfile                # python:3.11-slim, pre-cache embed model, uvicorn :8001
├── requirements.txt
├── app/
│   ├── main.py              # FastAPI service (4 endpoints + lifespan)
│   ├── download_model.py    # pre-cache โมเดลตอน Docker build
│   ├── _perf_probe.py       # เครื่องมือวัดเวลาแต่ละ node (throwaway)
│   ├── test_agent_flow.py   # สคริปต์ทดสอบ follow-up loop
│   └── RAG/
│       ├── __init__.py      # re-export public API
│       └── GraphRAG/
│           ├── config.py    # ค่าคอนฟิกทั้งหมด + sep()
│           ├── models.py    # Pydantic models ของ STIX entities/relationships
│           ├── main.py      # CLI (--ingest/--test/--agent/--retrieve-only)
│           ├── pipeline/    # agent_graph, chain, router, cross_lingual,
│           │               # query_decomposer, evaluator, query_merger,
│           │               # context_builder, report_generator, thanoy_client
│           ├── retrieval/   # vector_retriever, graph_retriever, reranker, hybrid_retriever
│           ├── ingestion/   # stix_parser, graph_loader, vector_loader
│           └── evaluation/  # ground_truth, retriever_metrics, generation_metrics,
│                           # eval_runner, crosslingual_benchmark,
│                           # generate_eval_dataset, test_metrics
├── docs/
│   └── _build_pdf.py        # render RAG_Module.md → HTML (mermaid) → PDF
└── finetune/
    ├── ft_config.py        # MITRE specialist fine-tune (QLoRA/LoRA)
    ├── data/templates.py   # STIX → (Q,A) templates
    ├── data/build_dataset.py # STIX → SFT jsonl (closed-book + grounded + abstention)
    ├── train/train_unsloth.py # LoRA trainer (Unsloth)
    ├── compare/run_comparison.py # A/B base vs fine-tune (env-swap)
    └── export/merge_and_gguf.py # merge LoRA → GGUF (llama.cpp)
```

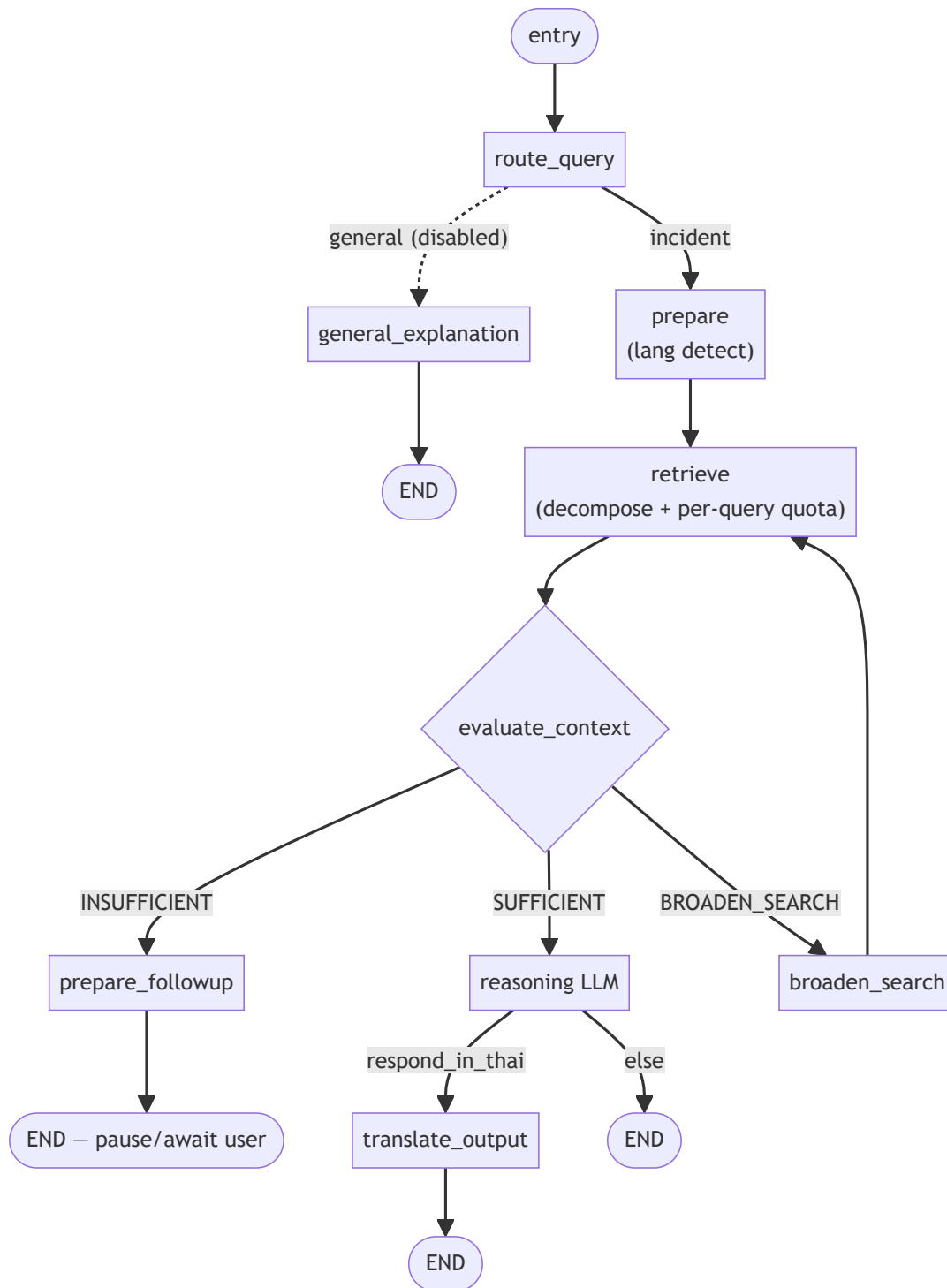
6. Request Lifecycle

6.1 Startup (lifespan)

โหลด BGE-M3 ครั้งเดียว → สร้าง `GraphRAGChain`, `GraphRAGAgent`, `HybridRetriever`, `ReportGenerator` (แชร์ embed model) → เชื่อม Neo4j/Qdrant → เก็บไว้ใน `app.state`. ถ้าพังจะ set เป็น `None` (endpoint คืน 503)

6.2 POST /query (agent)

`route_query` → `prepare` (ตรวจภาษา) → `retrieve` (decompose → `retrieve_multi_quota` → `build_context`) → `evaluate_context` → - **SUFFICIENT** → `reasoning` → (ถ้าไทย) `translate_output` → END - **INSUFFICIENT** → `prepare_followup` → END (pause, คืน `status="followup" + session_id`) - **BROADEN** → `broaden_search` → วง `retrieve`



resume: `/resume` ดึง state ที่ pause → `_resume_with_answer` (เก็บ slot + merge query + rewrite) → invoke graph ใหม่จากต้น (route → ...) จนถึง END

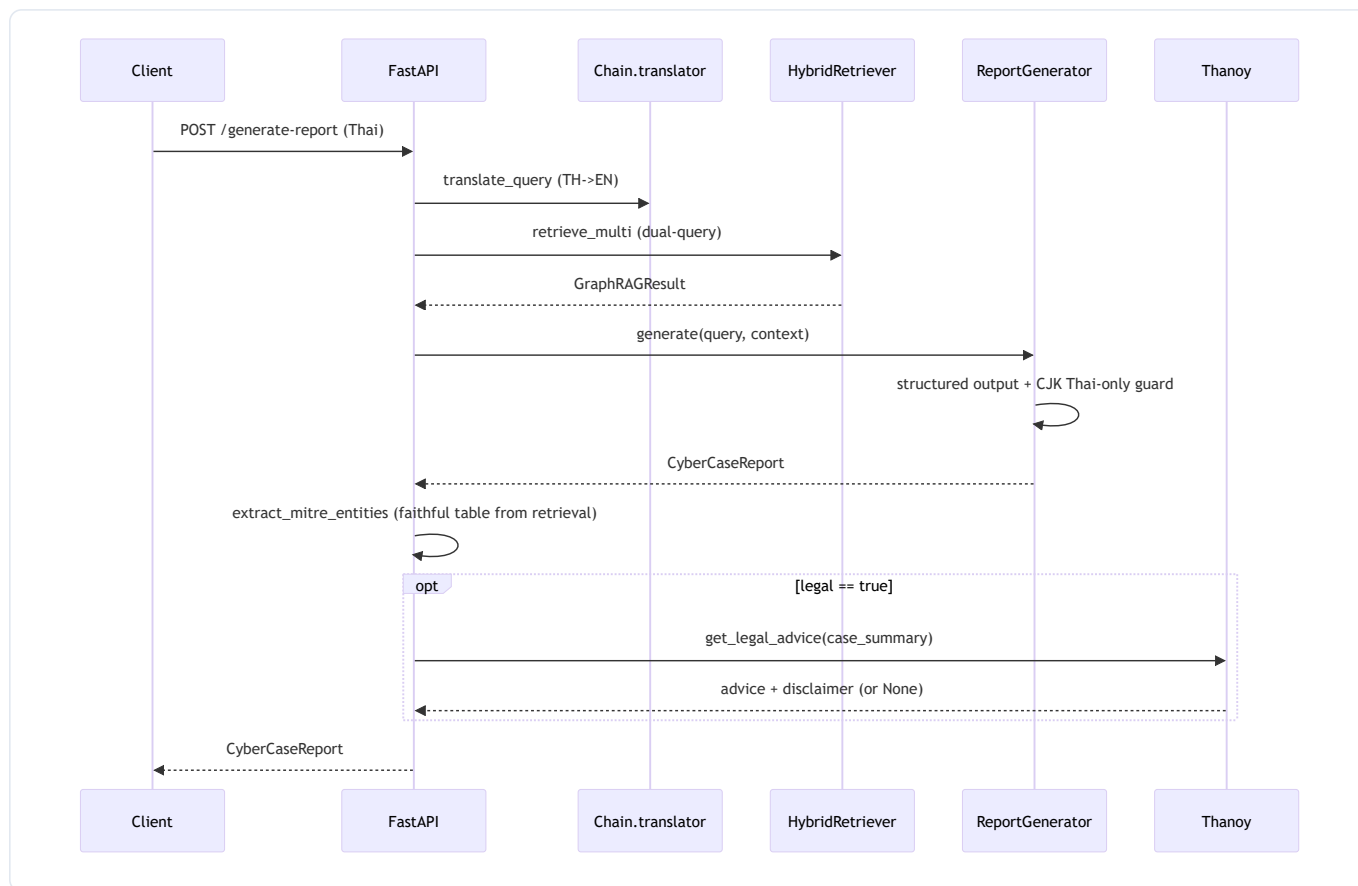
6.3 POST /resume

ดึง state ที่ pause จาก `_sessions[session_id]` → `_resume_with_answer` (เก็บ fact ลง slot, merge query, append rewrite, เพิ่ม `retry_count`) → invoke graph ใหม่จนจบ → คืน `status="completed"`

6.4 POST /generate-report

`translator.translate_query` (TH→EN) → `build_retrieval_queries` (dual-query) → `retrieve_multi` → `build_context` → `report_gen.generate` (structured output 7 ส่วน + CJK guard) → `extract_mitre_entities` (เขียนทับตารางด้วย entity จริง) → (ถ้า

`legal=true`) `get_legal_advice` (Thanoy)



7. อ้างอิงโค้ดทุกฟังก์ชัน

รูปแบบ: ชื่อ(พารามิเตอร์สำคัญ) — หน้าที่ ฟังก์ชัน private ขึ้นต้น `_` คือ helper ภายในไฟล์

7.1 `app/main.py` — FastAPI service

สัญลักษณ์	หน้าที่
<code>lifespan(app)</code> (<i>async ctx</i>)	โหลด BGE-M3 ครั้งเดียว, สร้าง chain/agent/retriever/report_gen ตาม <code>USE_LOCAL</code> , เก็บใน <code>app.state</code> ; ตอน shutdown ปิดทุกตัว
<code>QueryRequest</code>	request body: <code>query</code> , <code>use_agent=True</code> , <code>legal=False</code>
<code>QueryResponse</code>	response: <code>status</code> , <code>answer</code> , <code>followup_question</code> , <code>session_id</code>
<code>ResumeRequest</code>	request body: <code>session_id</code> , <code>answer</code>
<code>health(request)</code> (<i>async</i>)	คืนสถานะบริการ + ว่า chain/agent โหลดสำเร็จไหม
<code>query_rag(request, req)</code> (<i>async</i>)	endpoint <code>/query</code> : ถ้า <code>use_agent</code> เรียก <code>rag_agent.query</code> ไม่งั้น <code>rag_chain.query</code> ; map error → HTTP 500/503
<code>resume_agent(request, req)</code> (<i>async</i>)	endpoint <code>/resume</code> : เรียก <code>rag_agent.resume(session_id, answer)</code> ; KeyError → 404
<code>generate_report(request, req)</code> (<i>async</i>)	endpoint <code>/generate-report</code> : translate → retrieve_multi → build_context → generate → overwrite <code>mitre_entities</code> ด้วย retrieval จริง → (optional) Thanoy legal
<code>__main__</code>	<code>uvicorn.run(app, host=0.0.0.0, port=8001)</code>

7.2 config.py

ค่าคอนฟิกทั้งหมด (โหลด .env ด้วย python-dotenv) : - Paths: _SCRIPT_DIR , _PROJECT_ROOT , _STIX_DATA_DIR , ENTERPRISE/MOBILE/ICS_ATTACK_DIR - Embedding: EMBED_MODEL="BAAI/bge-m3" , EMBED_DIM=1024 , USE_FP16=True - Qdrant: QDRANT_HOST/PORT/API_KEY/URL , QDRANT_COLLECTION_ENTITIES/RELATIONSHIPS - RRF: RRF_K=60 , DENSE_WEIGHT=1.0 , SPARSE_WEIGHT=1.0 - Neo4j: NEO4J_URI/USER/PASSWORD - LLM: ANTHROPIC_API_KEY , LLM_MODEL="claude-haiku-4-5" , LLM_MAX_TOKENS=4096 , LLM_TEMPERATURE=0 , EVALUATOR_* , RAGAS_LLM_MODEL , OPENROUTER_* - Thanoy: THANOY_API_KEY/URL/TIMEOUT/ENABLED - Local (Ollama): OLLAMA_BASE_URL , LOCAL_LLM_MODEL="qwen2.5:7b" , LOCAL_EVAL_MODEL="gemma3:4b" , USE_LOCAL , LOCAL_NUM_CTX=8192 - Retrieval: VECTOR_TOP_K=10 , GRAPH_EXPANSION_DEPTH=2 , FINAL_TOP_K=5 , RERANKER_MODEL , ATTACK_DOMAIN_FILTER="enterprise" , DUAL_QUERY_RETRIEVAL - Domains: ATTACK_DOMAINS={"enterprise":..., "mobile":...}

ฟังก์ชัน	หน้าที่
sep(title="")	พิมพ์เส้นคั่น 72 ตัวอักษรพร้อมหัวข้อ (ใช้ทั่ว pipeline สำหรับ verbose log)

7.3 models.py — Pydantic STIX models

คลาส	หน้าที่
AttackEntity	base ของทุก entity: stix_id , attack_id , name , description , node_label , url , domain
Technique(AttackEntity)	+ platforms , is_subtechnique , tactics (kill-chain phase names)
Group(AttackEntity)	+ aliases (node_label="Group")
Software(AttackEntity)	+ aliases , software_type (tool / malware)
Campaign(AttackEntity)	+ aliases
Mitigation(AttackEntity)	node_label="Mitigation"
Tactic(AttackEntity)	+ shortname
DataSource(AttackEntity)	+ platforms
DataComponent(AttackEntity)	node_label="DataComponent"
AttackRelationship	edge: stix_id , relationship_type , source_ref , target_ref , source_name , target_name , description , edge_label

7.4 pipeline/

agent_graph.py — LangGraph agentic pipeline

สัญลักษณ์	หน้าที่
<code>AgentState</code> (<i>TypedDict</i>)	state ที่ไหลผ่านทุก node: inputs, route, english_query, graphrag_result, context, evaluation, retry/followup/broaden counts, incident_facts, asked_slots, strategy, answer ฯลฯ
<code>AgentResponse</code> (<i>dataclass</i>)	ผลลัพธ์: <code>status</code> ("completed"/"followup"), <code>answer</code> , <code>followup_question</code> , <code>session_id</code>
<code>AgentResponse.needs_followup</code> (<i>property</i>)	True ถ้า <code>status=="followup"</code>
<code>AgentResponse.to_dict()</code>	serialize เป็น dict สำหรับ JSON API
<code>MAX_RETRIEVAL_RETRIES=2</code> , <code>MAX_FOLLOWUP_RETRIES=2</code>	เพดานรอบ self-reflection / follow-up
<code>GraphRAGAgent.__init__(embed_model, reranker, use_local)</code>	โหลด/รับ embed model, สร้าง retriever + router + evaluator + query_merger + decomposer + reasoning/translation LLM, build graph, สร้าง <code>_sessions</code>
<code>.close()</code>	ปิด retriever (Neo4j + Qdrant)
<code>.retrieve_only(user_query)</code>	เฉพาะ retrieval: decompose → <code>retrieve_multi_quota</code> → <code>build_context</code> (debug)
<code>.query(user_query, verbose, followup_callback)</code>	รัน graph; ถ้า pause: CLI โหมดเรียก callback วน, API โหมดเก็บ session แล้วคืน <code>status="followup"</code>
<code>.resume(session_id, user_answer, verbose)</code>	ดึง session ที่ pop ออก → resume ด้วยคำตอบ → คืน completed (KeyError ถ้าไม่พบ)
<code>._resume_with_answer(state, user_answer)</code>	เก็บ answer ลง slot ที่ขาด, <code>query_merger.merge</code> ได้ rewrite, append <code>rewritten_queries</code> , เพิ่ม retry/followup count, invoke graph ใหม่
<code>._force_continue(state)</code>	ข้าม follow-up: เคลียร์ evaluation บังคับ path SUFFICIENT แล้ว invoke graph
<code>._build_graph()</code>	สร้าง <code>StateGraph</code> : register node, ตั้ง entry <code>route_query</code> , ผูก conditional/normal edges, <code>compile()</code>
<code>._node_route_query(state)</code>	เรียก <code>router.route_query</code> → GENERAL/INCIDENT
<code>._node_general_explanation(state)</code>	ตอบความรู้ทั่วไปด้วย LLM ตรง (ไม่ retrieve), ถ้าไทยเดิม "Answer in Thai"
<code>._node_prepare(state)</code>	ตรวจว่าตอบไทยไหม (<code>should_respond_in_thai</code>), set <code>english_query = query</code> (ไม่แปล input)
<code>._node_retrieve(state)</code>	full query เป็น channel แรก + decompose sub-queries + rewrites → <code>retrieve_multi_quota</code> → <code>build_context</code>
<code>._node_evaluate_context(state)</code>	เรียก <code>evaluator.evaluate</code> พร้อม incident_facts/asked_slots/total_retries → set evaluation/strategy/gap/ack
<code>._node_prepare_followup(state)</code>	ตั้ง <code>awaiting_followup=True</code> + <code>followup_question</code>
<code>._node_broaden_search(state)</code>	append <code>new_query</code> จาก evaluation, เพิ่ม broaden_count แล้ววน retrieve
<code>._node_reasoning(state)</code>	reasoning LLM → คำตอบอังกฤษ (มี fast-path ACKNOWLEDGE_LIMIT เมื่อ retry หมด); ใช้ <code>build_generation_prompt</code> + <code>get_reasoning_system_prompt</code>
<code>._node_translate_output(state)</code>	translation LLM → ไทย (ใช้ <code>get_translation_system_prompt</code>)
<code>._edge_after_route(state)</code> (<i>static</i>)	ปัจจุบันคืน "incident" เสมอ (router ถูกปิดชั่วคราว)
<code>._edge_after_evaluation(state)</code> (<i>static</i>)	SUFFICIENT→reasoning; ถึงเพดาน+BROADEN(<2)→broaden, ไม่งั้น→sufficient; ไม่งั้น→followup
<code>._edge_after_reasoning(state)</code> (<i>static</i>)	ถ้า <code>respond_in_thai</code> →translate ไม่งั้น→done

`chain.py` — Linear LCEL pipeline

สัญลักษณ์	หน้าที่
<code>_print_sources(graphrag_result, top_n=5)</code>	พิมพ์ top source (name/type/attack_id/score) สำหรับ verbose
<code>GraphRAGChain.__init__(embed_model, use_local)</code>	โหลด embed model, สร้าง translator(<code>CrossLingualLayer</code>) + retriever + router + reasoning/translation LLM
<code>.close()</code>	ปิด retriever
<code>.query(user_query, verbose, followup_callback)</code>	linear: route → (general?) → translate → dual-query retrieve → build_context → reasoning LLM → (ถ้าไทย) translation LLM; คืน string
<code>.retrieve_only(user_query)</code>	translate → <code>build_retrieval_queries</code> → <code>retrieve_multi</code> → <code>build_context</code> (debug)

router.py

สัญลักษณ์	หน้าที่
<code>ROUTER_SYSTEM_PROMPT</code>	prompt จัดประเภท GENERAL_EXPLANATION vs INCIDENT_ANALYSIS
<code>QueryRouter.__init__(use_local)</code>	สร้าง LLM (Claude/Ollama, max 32 tokens)
<code>.route_query(query)</code>	คืน label; ไม่มี LLM → fallback "INCIDENT_ANALYSIS"

cross_lingual.py

สัญลักษณ์	หน้าที่
<code>TRANSLATE_TO_ENGLISH_PROMPT</code>	prompt แปล Thai→EN คงศัพท์เทคนิค/ATT&CK ID
<code>REASONING_SYSTEM_PROMPT</code>	system prompt stage 2: simplify jargon, EN-only, 4 หัวข้อ (INCIDENT SUMMARY/ATTACK SEQUENCE/TECHNIQUES/IMPACT)
<code>TRANSLATE_TO_THAI_SYSTEM_PROMPT</code>	system prompt stage 3: EN→Thai คง ATT&CK ID/ชื่อ
<code>_is_thai(text)</code>	มีอักษรไทย (๐-๙) ไหม
<code>build_retrieval_queries(original, english, extra=None)</code>	สร้าง list query: อังกฤษก่อน + (dual-query) ไทยต้นฉบับ + rewrites
<code>_is_mostly_english(text)</code>	สัดส่วน ASCII-alpha > 70% ไหม
<code>CrossLingualLayer.__init__(use_local)</code>	สร้าง translate LLM (256 tokens) หรือ None ถ้าไม่มี key
<code>.translate_query(query)</code>	Thai→EN; ถ้าเป็นอังกฤษอยู่แล้ว/ไม่มี LLM คืนเดิม
<code>.get_reasoning_system_prompt() (static)</code>	คืน <code>REASONING_SYSTEM_PROMPT</code>
<code>.get_translation_system_prompt() (static)</code>	คืน <code>TRANSLATE_TO_THAI_SYSTEM_PROMPT</code>
<code>.should_respond_in_thai(query) (static)</code>	= <code>_is_thai(query)</code>

query_decomposer.py

สัญลักษณ์	หน้าที่
<code>_SYSTEM</code>	prompt แยก incident เป็น sub-query atomic เชิงเหตุการณ์ (ภาษาเดิม, ครอบคลุม kill-chain stage รวม exfil/impact, ห้ามคำหยาบคาย)
<code>_MAX_SUBQUERIES=10</code>	เพดานจำนวน sub-query (ตั้งสูงพอครอบคลุม kill chain)
<code>_parse(text, cap)</code>	แปลง output ที่ละบรรทัด → list ที่ strip bullet/เลข, dedup, cap
<code>QueryDecomposer.__init__(use_local)</code>	สร้าง LLM (Claude 512 tokens / Ollama reasoning=False) หรือ None
<code>.decompose(incident, max_subqueries, verbose)</code>	คืน sub-query atomic; error/ไม่มี LLM → fallback <code>[incident]</code>

evaluator.py

สัญลักษณ์	หน้าที่
<code>VERDICT_SUFFICIENT/INSUFFICIENT/NEED_CLARIFICATION</code> , <code>VALID_VERDICTS</code> , <code>MAX_RETRIES=2</code>	ค่าคงที่ verdict
<code>EvaluationResult</code> (<i>dataclass</i>)	verdict, reason, covered/missing_phases, missing_slot, follow_up, strategy, new_query, gap_warning, message
<code>EvaluationResult.__post_init__</code>	ตั้ง list ว่าถ้า None
<code>EVALUATOR_SYSTEM_PROMPT</code>	prompt ประเมินความพอเพียงตาม 4 phase + slot logic + fallback strategy; ห้ามแต่ง ATT&CK ID ที่ไม่อยู่ใน context
<code>ContextEvaluator.__init__(use_local)</code>	สร้าง evaluator LLM (Claude Haiku/Gemma)
<code>.evaluate(original_query, english_query, context, retry_count, ..., incident_facts, asked_slots)</code>	ประเมิน; ถ้า <code>retry_count ≥ MAX_RETRIES</code> short-circuit SUFFICIENT; หา next missing slot; เรียก LLM → parse
<code>._build_prompt(...)</code> (<i>static</i>)	ประกอบ prompt: known facts + asked slots + retry hint + query + context (ตัด 4000 ตัวอักษร)
<code>._parse_response(raw)</code> (<i>static</i>)	ดึง JSON (regex → brace-scan → fallback SUFFICIENT) เป็น <code>EvaluationResult</code>

query_merger.py

สัญลักษณ์	หน้าที่
<code>_MERGE_PROMPT_TEMPLATE</code>	prompt รวม query เดิม + follow-up answer → query เดี่ยว MITRE-aligned, self-contained, EN-only
<code>QueryMerger.__init__(use_local)</code>	LLM น้ำหนักเบา (evaluator model)
<code>.merge(original_query, followup_question, user_answer, verbose)</code>	คืน query รวม; ไม่มี LLM → concatenate ธรรมดา

context_builder.py

ฟังก์ชัน	หน้าที่
<code>build_context(result, max_context_length=10000, max_vector=None, max_graph=3)</code>	ประกอบ context string: semantic results (top <code>max_vector</code> หรือ <code>FINAL_TOP_K</code>) + graph subgraph (<code>max_graph</code>); ตัดความยาว
<code>build_generation_prompt(context, original_query, english_query, respond_in_thai, incident_facts)</code>	ประกอบ user prompt: confirmed facts (priority สูง) + context + คำถาม + instruction (ไทย/อังกฤษ)

report_generator.py

สัญลักษณ์	หน้าที่
<code>_CJK_RE</code>	regex จับอักษรจีน/ญี่ปุ่น/เกาหลี (กัน code-switch)
<code>MitreEntity</code> (<i>pydantic</i>)	1 แถวตาราง MITRE: <code>id</code> , <code>name</code> , <code>type</code>
<code>CyberCaseReport</code> (<i>pydantic</i>)	รายงาน 7 ส่วน: <code>case_summary</code> , <code>detected_indicators</code> , <code>mitre_mapping</code> , <code>mitre_entities</code> , <code>mapping_justification</code> , <code>evidence_to_investigate</code> , <code>preliminary_recommendations</code> , <code>system_limitations</code> , + <code>legal_advice</code> (optional)
<code>_MAPPING_TABLE_TYPES</code> , <code>_MAPPING_TABLE_MAX=25</code>	type ที่อนุญาตในตาราง (Technique/Subtechnique/Tactic) + เพดานแถว
<code>ReportGenerator.__init__(use_local)</code>	สร้าง LLM (Claude/Ollama) + <code>with_structured_output(CyberCaseReport)</code> + prompt (Thai-only constraint)
<code>.generate(query, context)</code>	invoke structured chain → <code>_sanitize_thai</code>
<code>._rewrite_to_thai(text)</code>	ถ้า field มี CJK: เรียก LLM แปลเป็นไทยล้วน; fallback strip CJK
<code>._sanitize_thai(report)</code>	สแกนทุก field (str + list[str]) ถ้าเจอ CJK → repair
<code>extract_mitre_entities(rag_result, max_rows=25)</code>	สร้างตารางจาก vector hits + graph center nodes เท่านั้น (ไม่เอา neighbors), filter เฉพาะ Technique/Subtechnique/Tactic, dedup by ID

thanoy_client.py

สัญลักษณ์	หน้าที่
<code>LEGAL_DISCLAIMER</code>	ข้อความ disclaimer ไทย (AI ไม่ผูกพันทางกฎหมาย)
<code>_QUERY_TEMPLATE</code>	prompt ถาม Thanoy ว่าเข้าข่ายกฎหมายไทยฉบับ/มาตราใด + ประเมินความเสียหาย
<code>_build_query(case_summary)</code>	ใส่ summary ลง template
<code>_parse_response(data)</code>	ดึงข้อความ advice จาก response (รองรับหลาย shape)
<code>get_legal_advice(case_summary, timeout)</code> (<i>async</i>)	เรียก Thanoy REST; คืน advice+disclaimer หรือ None (ไม่มี key/ว่าง/error) — ไม่เคย raise

7.5 retrieval/

vector_retriever.py

สัญลักษณ์	หน้าที่
<code>VectorResult</code> (<i>dataclass</i>)	ผล vector: <code>document</code> , <code>metadata</code> , <code>score</code> , <code>stix_id</code>
<code>VectorRetriever.__init__(embed_model)</code>	เชื่อม Qdrant (cloud/local/in-memory), โหลด/รับ embed model, พิมพ์จำนวน docs
<code>._search_hybrid(collection, query, top_k, qdrant_filter)</code>	embed dense+sparse → Qdrant <code>query_points</code> (Prefetch dense + sparse, RRF fusion) → <code>VectorResult[]</code>
<code>.search_entities(query, top_k, node_label_filter)</code>	ค้น entities; over-fetch x3 แล้วกรอง <code>domain==ATTACK_DOMAIN_FILTER</code> ใน Python (กัน mobile)
<code>.search_relationships(query, top_k, edge_label_filter)</code>	ค้น relationships (มี edge_label filter)
<code>._normalize_scores(results)</code> (<i>static</i>)	min-max normalize คะแนนใน list (ให้เทียบข้าม collection ได้)
<code>.search_all(query, top_k)</code>	ค้นทั้ง entities (full quota) + relationships (ครั้ง) → normalize → merge → sort → top_k

graph_retriever.py

สัญลักษณ์	หน้าที่
<code>GraphNode</code> (<i>dataclass</i>)	node กราฟ: <code>stix_id</code> , <code>name</code> , <code>label</code> , <code>attack_id</code> , <code>description</code>
<code>GraphEdge</code> (<i>dataclass</i>)	edge: <code>edge_label</code> , <code>source_name</code> , <code>target_name</code> , <code>description</code>
<code>SubgraphResult</code> (<i>dataclass</i>)	<code>center_node</code> , <code>neighbors</code> , <code>edges</code>
<code>SubgraphResult.to_text()</code>	render subgraph เป็นข้อความ (จัดกลุ่มตาม edge type, map ชื่อแสดงผล เช่น USES→"Used by")
<code>GraphRetriever.__init__()</code>	เชื่อมต่อ Neo4j driver
<code>.close()</code>	ปิด driver
<code>.expand(stix_ids)</code>	ขยาย subgraph ของแต่ละ id (dedup) → <code>SubgraphResult[]</code>
<code>._expand_single(stix_id)</code>	ดึง center node + outgoing + incoming relationships (Cypher)
<code>.query_cypher(cypher, params)</code>	รัน Cypher ใดๆ คืน list[dict]
<code>.get_multi_hop_path(start_name, end_name, max_hops=4)</code>	หา shortestPath ระหว่าง 2 entity แบบ format อ่านง่าย

reranker.py

สัญลักษณ์	หน้าที่
<code>Reranker.__init__(model_name=RERANKER_MODEL)</code>	โหลด <code>CrossEncoder</code> (bge-reranker-v2-m3, max_length 512)
<code>.rerank(query, results, top_k)</code>	ให้คะแนน (query, doc) ใหม่ → sigmoid เข้า [0,1] → sort → พิมพ์ top-K, คืน reranked

hybrid_retriever.py

สัญลักษณ์	หน้าที่
<code>GraphRAGResult</code> (<i>dataclass</i>)	<code>vector_results</code> , <code>graph_results</code>
<code>GraphRAGResult.get_context_text(max_length=8000)</code>	format ผลรวมเป็นข้อความ (legacy helper)
<code>_TYPE_WEIGHTS</code>	ตัวคูณคะแนนตาม node type (Technique×1.2, Tactic×1.1, Group×0.75, Software×0.8, Campaign×0.75)
<code>HybridRetriever.__init__(embed_model, reranker)</code>	สร้าง VectorRetriever + GraphRetriever + Reranker
<code>.close()</code>	ปิด Neo4j + Qdrant client
<code>._reweight_by_type(vector_results)</code> (<i>static</i>)	คูณคะแนนตาม type แล้ว re-sort (technique ลอยขึ้น, graph seed เปลี่ยนตาม)
<code>.retrieve(query, top_k, node_label_filter)</code>	vector search → rerank → reweight → ดึง stix_id (ตามลำดับ relevance) → graph expand → <code>GraphRAGResult</code>
<code>.retrieve_multi(queries, top_k, node_label_filter)</code>	รัน <code>retrieve</code> ต่อ query แล้ว merge: vector เก็บคะแนน สูงสุดต่อ id, graph เก็บ subgraph แรกต่อ center; re-sort
<code>.retrieve_multi_quota(queries, per_query_k=3, top_k, max_vector=15, max_graph=8, node_label_filter)</code>	เก็บ top- <code>per_query_k</code> ของแต่ละ query → round-robin interleave (ทุก sub-query ได้พื้นที่ในต้น list) → cap

7.6 ingestion/

stix_parser.py

สัญลักษณ์	หน้าที่
<code>_get_attack_id(obj)</code>	ดึง ATT&CK ID จาก <code>external_references</code>
<code>_get_url(obj)</code>	ดึง URL จาก <code>external_references</code>
<code>_is_revoked_or_deprecated(obj)</code>	True ถ้า revoked/deprecated (กรองทิ้ง)
<code>_get_tactics_from_kill_chain(obj)</code>	ดึง tactic shortnames จาก <code>kill_chain_phases</code>
<code>RELATIONSHIP_TYPE_MAP</code> , <code>STIX_TYPE_TO_LABEL</code>	map STIX type → edge label / node label
<code>StixParser.__init__()</code>	init list entities/relationships + lookup tables
<code>.parse_folder(folder, domain)</code>	parse ไฟล์ <code>.json</code> ทั้งหมดในโฟลเดอร์
<code>.parse_file(filepath, domain)</code>	parse 1 bundle: pass 1 สร้าง entities, pass 2 relationships, + derived edges, สรุปจำนวน
<code>._parse_technique/group/software/campaign/mitigation/tactic/data_source/data_component(obj, ...)</code>	สร้าง entity แต่ละชนิดจาก STIX object
<code>._build_relationships(raw_rels)</code>	สร้าง <code>AttackRelationship</code> จาก raw STIX (เฉพาะ endpoint ที่มีจริง)
<code>._build_tactic_edges()</code>	derive <code>IN_TACTIC</code> จาก technique kill_chain_phases
<code>._build_data_source_edges()</code>	derive <code>HAS_COMPONENT</code> จาก <code>x_mitre_data_source_ref</code>
<code>.get_entities_by_label(label)</code> / <code>.get_relationships_by_label(label)</code>	filter ตาม label
<code>parse_all_domains()</code>	parse ทุก domain ใน <code>ATTACK_DOMAINS</code> , dedup entities/relationships

graph_loader.py

สัญลักษณ์	หน้าที่
<code>GraphLoader.__init__()</code>	เชื่อม Neo4j
<code>.close()</code>	ปิด driver
<code>.clear_database()</code>	<code>MATCH (n) DETACH DELETE n</code>
<code>.create_constraints()</code>	สร้าง unique constraint บน <code>stix_id</code> ทุก label + <code>:Entity</code>
<code>.create_indexes()</code>	สร้าง index (attack_id/name/shortname)
<code>.load_entities(entities)</code>	UNWIND batch MERGE node ตาม label + เพิ่ม <code>:Entity</code>
<code>._entity_to_props(entity)</code>	แปลง entity → property dict (ตัด description 5000)
<code>.load_relationships(relationships)</code>	UNWIND batch CREATE edge ตาม edge_label (match ผ่าน <code>:Entity</code>)
<code>.load_all(parser)</code>	clear → constraints → indexes → nodes → edges → พิมพ์สถิติ

vector_loader.py

สัญลักษณ์	หน้าที่
<code>uuid_from_stix_id(stix_id)</code>	แปลง stix_id → UUID ที่ valid (ใช้เป็น point id)
<code>VectorLoader.__init__(embed_model)</code>	เชื่อม Qdrant + โหลด/รับ embed model
<code>._embed_texts(texts)</code>	embed batch → dense + sparse (lexical_weights)
<code>._init_collection(name)</code>	สร้าง collection (ลบของเดิมก่อน) ด้วย dense(1024,Cosine)+sparse
<code>.load_entities(entities)</code>	embed <code>"{label}: {name}. {desc}"</code> + payload (รวม <code>domain</code>) → upsert batch
<code>.load_relationships(relationships)</code>	embed <code>"{src} {edge} {tgt}: {desc}"</code> + payload → upsert batch
<code>.load_all(parser)</code>	embed entities + relationships

7.7 evaluation/

ground_truth.py

สัญลักษณ์	หน้าที่
<code>EvalSample (dataclass)</code>	<code>query</code> , <code>relevant_stix_ids</code> , <code>reference_answer</code> , <code>language</code> , <code>category</code>
<code>EvalSample.has_reference_answer()</code>	มี reference answer ไหม
<code>load_ground_truth(path)</code>	โหลด dataset JSON → <code>EvalSample[]</code>
<code>save_ground_truth(samples, path)</code>	เซฟ <code>EvalSample[]</code> → JSON

retriever_metrics.py

ฟังก์ชัน	หน้าที่
<code>hit_at_k(retrieved, relevant, k)</code>	มี relevant ใน top-K ไหม (1/0)
<code>recall_at_k(...)</code>	สัดส่วน relevant ที่เจอใน top-K
<code>precision_at_k(...)</code>	สัดส่วน top-K ที่ relevant
<code>reciprocal_rank(retrieved, relevant)</code>	1/rank ของ relevant ตัวแรก
<code>ndcg_at_k(...)</code>	NDCG@K (binary relevance)
<code>average_precision(...)</code>	Average Precision
<code>RetrieverEvalResult (dataclass)</code>	metric รวม (per-K dict + MRR/MAP + latency)
<code>RetrieverEvalResult.to_table()</code>	format ตารางผล
<code>evaluate_retriever(retriever_fn, samples, k_values, name)</code>	รัน retriever ต่อทุก sample, วัด metric (มี inner <code>mean()</code>)

generation_metrics.py

สัญลักษณ์	หน้าที่
<code>_tokenize(text)</code>	whitespace+lowercase tokenizer
<code>token_f1(prediction, reference)</code>	precision/recall/f1 ระดับ token
<code>rouge_l(prediction, reference)</code>	ROUGE-L (LCS)
<code>_try_ragas_evaluate(questions, answers, contexts, refs, use_local)</code>	รัน RAGAS (faithfulness + answer_correctness) เลือก judge Claude→OpenRouter, embeddings = nomic local; None ถ้าไม่มี
<code>_try_bertscore(predictions, references)</code>	BERTScore F1 (None ถ้าไม่ติดตั้ง)
<code>GenerationEvalResult (dataclass)</code>	RAGAS + fallback metrics + latency + per_sample
<code>GenerationEvalResult.to_table()</code>	format ตาราง
<code>evaluate_generation(query_fn, samples, use_local)</code>	รัน generation ต่อ sample, fallback metrics, RAGAS (มี inner <code>_safe_mean</code>)

eval_runner.py

สัญลักษณ์	หน้าที่
<code>_make_vector_retriever_fn(embed_model)</code>	คืน <code>(fn, None)</code> — vector-only retriever
<code>_make_graph_retriever_fn()</code>	คืน <code>(fn, close)</code> — graph-only (Cypher keyword/attack-id search + 1-hop)
<code>_collect_hybrid_ids(result)</code>	flatten <code>GraphRAGResult</code> → list stix_id (vector + center + neighbors, dedup)
<code>_make_hybrid_retriever_fn(embed_model)</code>	คืน <code>(fn, close)</code> — hybrid single-query
<code>_make_hybrid_quota_retriever_fn(embed_model, use_local)</code>	คืน <code>(fn, close)</code> — decompose + <code>retrieve_multi_quota</code> (mirror production agent)
<code>_make_generation_fn(embed_model, use_local)</code>	คืน <code>(fn, close)</code> — wrap <code>GraphRAGChain</code> คืน (answer, context_chunks)
<code>EvalRunner.__init__(dataset_path, mode, use_local, max_samples)</code>	โหลด dataset (กรอง >50 ids), cap samples
<code>._get_embed_model()</code>	lazy-load + share BGE-M3
<code>.run()</code>	รันตาม mode (retriever/generation/full) + cleanup
<code>._run_retriever_eval()</code>	benchmark 4 retriever (Vector/Graph/Hybrid/Hybrid+Quota) + comparison
<code>._run_generation_eval()</code>	benchmark generation
<code>._print_comparison(results)</code>	ตารางเทียบ retriever (K=5 + MRR/MAP + latency)
<code>main()</code>	CLI (<code>--dataset/--mode/--output/--max-samples/--local</code>), มี inner <code>Tee</code> (tee output ลงไฟล์)

crosslingual_benchmark.py

สัญลักษณ์	หน้าที่
<code>load_cache(path) / save_cache(cache, path)</code>	โหลด/เซฟ translation cache JSON
<code>translate_all(samples, cache_path, use_local)</code>	แปลทุก query ครั้งเดียว (cache + checkpoint)
<code>RetrievalBackend.__init__(with_graph, top_k)</code>	สร้าง stack รวม (embed + reranker + Qdrant หรือ full hybrid)
<code>.close()</code>	ปิด resource
<code>.retrieve_ids(queries)</code>	เลือก hybrid หรือ vector+rerank
<code>._retrieve_vector_rerank(queries)</code>	vector→rerank, merge max-score
<code>._retrieve_hybrid(queries)</code>	<code>retrieve_multi</code> + flatten ids
<code>print_comparison(results)</code>	ตารางเทียบ tRAG/Thai-direct/Dual-query
<code>main()</code>	CLI benchmark; inner <code>trag_fn / thai_direct_fn / dual_fn / Tee</code>

generate_eval_dataset.py

สัญลักษณ์	หน้าที่
<code>GeneratedSample (dataclass)+ .to_dict()</code>	sample ที่ gen (query/ids/answer/lang/category)
<code>Neo4jGroundTruthBuilder.__init__/.close/.run_query</code>	เชื่อม Neo4j + รัน Cypher
<code>.get_top_techniques/groups/software(limit)</code>	หา node ที่ degree สูง
<code>.get_all_tactics()</code>	ดึง tactics ทั้งหมด
<code>.get_groups_with_campaigns(limit)</code>	กลุ่มที่มี campaign attributed
<code>.get_techniques_with_detection(limit)</code>	technique ที่มี DataComponent detect
<code>.get_techniques_by_attack_ids(ids)</code>	map {attack_id: stix_id}
<code>QueryTemplateRegistry.__init__(neo4j)</code>	registry template query → Cypher
<code>.generate_mitigation_lookup / technique_lookup / group_software / group_techniques / tactic_techniques / software_techniques / technique_detection / technique_groups / software_type_query / campaign_attribution(...)</code>	สร้าง <code>GeneratedSample</code> 1 รายการ/template โดย ground truth มาจาก Cypher
<code>THAI_QUERY_TEMPLATES , THAI_ANSWER_PREFIX</code>	template ไทย deterministic
<code>_make_thai_variant(sample, seed_node)</code>	สร้าง variant ไทยจาก sample อังกฤษ
<code>INCIDENT_SCENARIOS</code>	scenario incident แบบ bilingual (กำกับ technique_ids + คำตอบ TH/EN)
<code>IncidentScenarioGenerator.__init__/.generate</code>	สร้าง incident samples (lookup stix_id จาก ATT&CK ID)
<code>DatasetGenerator.__init__(neo4j, thai_ratio)</code>	orchestrator
<code>.generate()</code>	วน template x seed node + incident + Thai variants (มี inner <code>_add</code> กัน dup/empty)
<code>ValidationResult (dataclass)+ .summary()</code>	ผล validate + รายงาน
<code>DatasetValidator.__init__/.validate(samples)</code>	ตรวจ: ไม่มี empty ids, ไม่มี query ซ้ำ, จำนวนขั้นต่ำ, ครอบคลุม category
<code>save_dataset(samples, path) / load_dataset_for_validation(path)</code>	I/O dataset
<code>main()</code>	CLI (<code>--output/--min-samples/--thai-ratio/--validate-only</code>)

test_metrics.py

`test_hit_at_k`, `test_recall_at_k`, `test_precision_at_k`, `test_reciprocal_rank`, `test_ndcg_at_k`, `test_average_precision`, `test_token_f1`, `test_rouge_l`, `test_ground_truth_io` — unit test ของ metric แต่ละตัว (ไม่พึ่ง DB); `run_all_tests()` รันทั้งหมด สรุป pass/fail

__init__.py

re-export `EvalSample`, `load_ground_truth`, `save_ground_truth`, `evaluate_retriever`, `evaluate_generation`

7.8 CLI & utilities

RAG/GraphRAG/main.py — CLI

สัญลักษณ์	หน้าที่
(UTF-8 fix)	reconfigure stdout/stderr เป็น utf-8 (Windows)
<code>run_ingest()</code>	parse STIX → โหลด Neo4j (<code>GraphLoader</code>) + Qdrant (<code>VectorLoader</code>)
<code>TEST_QUERIES</code>	ชุด query ไทยทดสอบ (~29 เคส)
<code>run_tests(retrieve_only, use_agent)</code>	รัน test queries ผ่าน chain/agent
<code>_interactive_followup_callback(question)</code>	callback อ่านคำตอบ follow-up จาก stdin (interactive)
<code>run_interactive(retrieve_only, use_agent)</code>	โหมด interactive REPL
<code>main()</code>	argparse: <code>--ingest/--test/--retrieve-only/--agent</code>

`download_model.py`

`download_model()` — pre-cache BGE-M3 + reranker (`mmarco-mMiniLMv2`) ตอน Docker build

`_perf_probe.py` — throwaway perf tool

`_record(label, dt)` สละเวลา · `timed(label)` decorator factory (มี inner `deco/wrapper`) · `main()` instrument ทุก node + retrieval substep แล้วรัน query เดียววัดเวลา

⚠️ อ้างถึง `_node_translate_query` ซึ่งถูก rename เป็น `_node_prepare` แล้ว — ต้องแก้ก่อนรัน

`test_agent_flow.py` — follow-up loop test

`TEST_CASES` (query คลุมเครือ + คำตอบจำลอง) · `make_callback(simulated_answer)` คืน callback ตอบอัตโนมัติ (inner `callback`) · `__main__` รันแต่ละเคสผ่าน `GraphRAGAgent.query`

`docs/_build_pdf.py` — เครื่องมือ doc (สคริปต์ระดับ module)

`_stash_mermaid(m)` แยก mermaid block ก่อนแปลง · `gh_slugify(value, separator)` slugify แบบ GitHub (คงไทย) · `_restore_mermaid(m)` ใส่ mermaid กลับเป็น `<pre class="mermaid">`; ส่วนล่างแปลง `RAG_Module.md` → HTML (Sarabun font + mermaid) เขียนไฟล์

7.9 finetune/

`ft_config.py`

ค่าคอนฟิก fine-tune: paths (`MODULE_DIR` , `RAG_PKG_ROOT` , `STIX_DOMAIN_DIRS`), held-out files, outputs (train/val/stats), models (`BASE_MODEL_HF="Qwen/Qwen3.5-4B"` , `FT_MODEL_OLLAMA="mitre-qwen3.5:4b"`), dataset knobs, system prompts (`SPECIALIST_SYSTEM_PROMPT` , `GROUNDING_SYSTEM_PROMPT`), training hyperparams (16-bit LoRA, `LORA_R=16` , `NUM_EPOCHS=1` ฯลฯ)

ฟังก์ชัน	หน้าที่
<code>add_rag_to_path()</code>	ใส่ <code>rag_service/app/RAG</code> ลง sys.path ให้ <code>import GraphRAG.*</code> ได้

`data/templates.py` — STIX → (Q,A) formatting

`clean_text(text, max_chars)` ลบ markdown noise/ตัดที่ขอบประโยค · `first_sentence(text, max_chars)` ประโยคแรกสมบูรณ์ · `_pick(rng, options)` สุ่มเลือก · `_join_list(items, max_items)` join + cap · `_ensure_period(s)` เติมจุด Template (คืน (q, a)): `technique_lookup` , `mitigation_lookup` , `technique_profile` , `technique_groups` , `technique_detection` , `tactic_techniques` , `group_techniques` , `group_software` , `software_techniques` , `software_type_query` , `campaign_attribution` Grounded helpers: `build_entity_context(...)` (semantic block), `build_relation_context(...)` (semantic+graph block), `grounded_list_answer(...)` (อ้างอิงเฉพาะ center ID + neighbor names), `abstention_answer(...)` (ตอบว่าไม่อยู่ใน context), `grounded_user_prompt(context, question)`

`data/build_dataset.py` — STIX → SFT jsonl

`_latest_bundle(folder)` หา STIX version ล่าสุด · `load_parser(domains, all_versions)` parse + dedup · `load_heldout_ids()` รวม stix_id จาก eval (กัน leak) · `build_indices(parser)` สร้าง relationship index (inner `label`) · `_record(...)` สร้าง record chat-format · `generate_examples(...)` วงสร้าง closed-book + grounded twin + abstention ต่อ technique/tactic/group/software/campaign (inner `add_grounding` / `add_abstention` / `ok`) · `dedup(records)` · `cap_per_category(records, max, rng)` · `write_jsonl(path, records)` · `main()` CLI build train/val + stats

`train/train_unsloth.py`

`main()` — โหลด base (Unsloth, 16-bit LoRA/4-bit QLoRA/full ตาม config) → attach LoRA → apply chat template (thinking off, inner `to_text`)
→ `SFTTrainer` + `train_on_responses_only` (mask prompt) → train → save adapter → (optional) export GGUF

`compare/run_comparison.py`

`run_eval(model, dataset, max_samples, out_md)` รัน eval generation ด้วย `LOCAL_LLM_MODEL=model` (stop ollama models กัน VRAM thrash) · `parse_metrics(md_path)` ดึง metric จากรายงาน · `render(...)` ตาราง A/B + Δ · `main()` รัน base + ft แล้วเทียบ

`export/merge_and_gguf.py`

`merge(base_model, adapter_dir, merged_dir)` โหลด base+LoRA → merge fp16 → save HF · `to_gguf(merged_dir, llama_cpp, quant)`
แปลง HF → GGUF + quantize ผ่าน llama.cpp · `main()` CLI merge (+optional GGUF)

ภาคผนวก: ข้อสังเกต/ข้อจำกัดที่ฝังในโค้ด

- **Router ถูกปิดชั่วคราว:** `_edge_after_route` คั้น "incident" เสมอ → ทุก query เข้า incident analysis (โค้ด GENERAL ถูก comment)
- `config.ATTACK_DOMAINS` ชี้ path STIX ได้ `rag_service/` ซึ่งไม่ตรงตำแหน่งจริง (`Mitre_ATT&CK Doc/` อยู่ที่ repo root) — `finetune/ft_config.py` resolve เองด้วย `STIX_DOMAIN_DIRS`
- **Domain filter** กรองได้เฉพาะ entity vector hits (relationships ไม่มี payload `domain`) → mobile ยังหลุดผ่าน graph center/relationship ได้บ้าง; แก้ 100% ต้อง re-ingest enterprise-only
- `/generate-report` ใช้เส้นทางเดิม (translate + dual-query + `retrieve_multi`) ต่างจาก agent path (decompose + quota)
- **CJK guard** เป็น belt-and-suspenders เพราะ prompt Thai-only อย่างเดียวกัน code-switch ของ Haiku ไม่อยู่
- `_perf_probe.py` ~~อ้าง node เดิมที่ rename แล้ว~~ → แก้แล้ว: ใช้ `_node_prepare` (เดิม `_node_translate_query`)
- `download_model.py` ~~cache reranker ตัวเก่า~~ → แก้แล้ว: cache `BAAI/bge-reranker-v2-m3` ให้ตรงกับ `RERANKER_MODEL` ที่ runtime ใช้